



"Año De La Recuperación Y Consolidación De La
Economía Peruana"



UNIVERSIDAD PERUANA LOS ANDES

"FACULTAD DE INGENIERÍA"

ESCUELA PROFESIONAL "SISTEMAS Y COMPUTACIÓN"

IMPLEMENTACIÓN Y ADMINISTRACIÓN DE LA BASE DE DATOS QHATUPERU EN SQL SERVER

CÁTEDRA: Base de Datos II

CATEDRÁTICO: Ing. Fernandez Bejarano Raul Enrique

ESTUDIANTE: FLORES ESTRELLA Wilson Samir

CICLO: V

SECCIÓN: A1

HUANCAYO PERÚ

2025

Ejercicio 1: Creación y distribución de archivos físicos de la base QhatuPeru

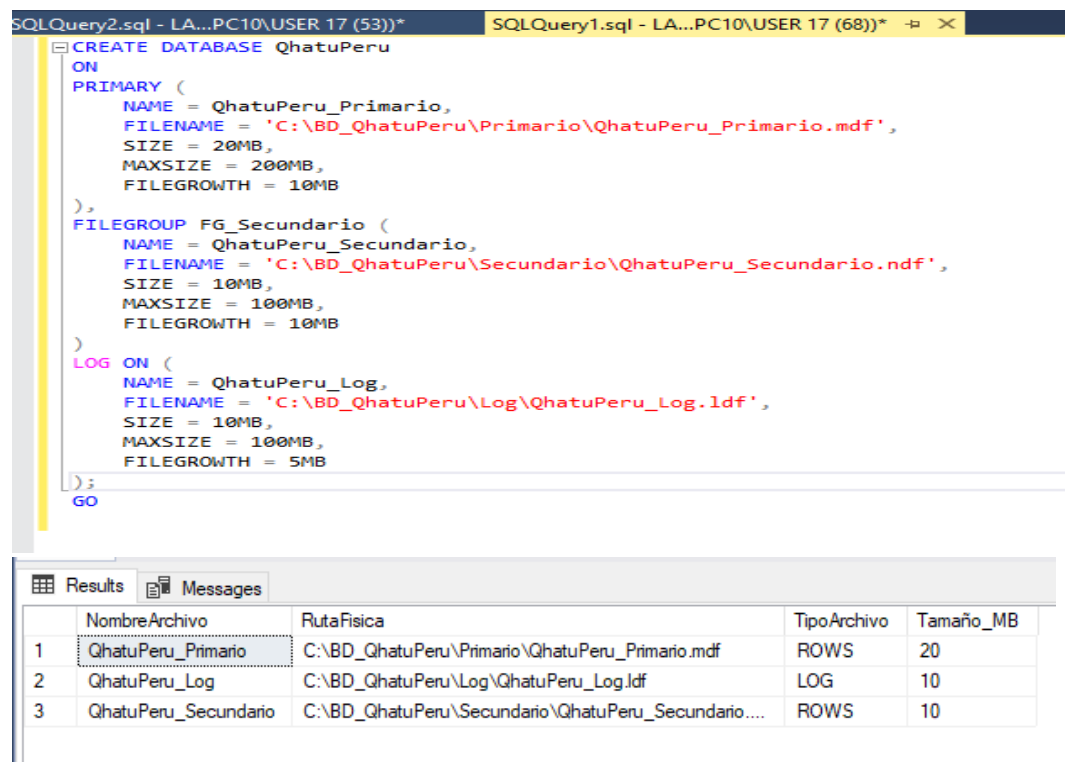
Enunciado del ejercicio

Como parte de la expansión comercial de la empresa **QhatuPeru**, se requiere la creación de una nueva base de datos que distribuya la información entre un archivo primario, un archivo secundario y un archivo de log de transacciones, con el objetivo de mejorar el rendimiento y la seguridad en el almacenamiento de datos.

Ejercicio práctico:

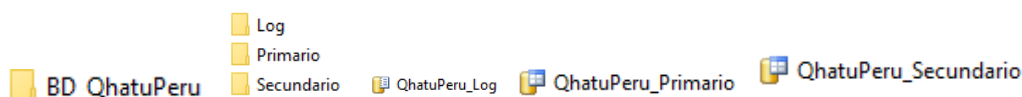
- a) Crear la base de datos **QhatuPeru** con un archivo primario, un archivo secundario y un archivo de log de transacciones en rutas distintas.
- b) Consultar los archivos físicos asociados a la base de datos **QhatuPeru**.

Script de la solución en T-SQL



```
CREATE DATABASE QhatuPeru
ON
PRIMARY (
    NAME = QhatuPeru_Primary,
    FILENAME = 'C:\BD_QhatuPeru\Primario\QhatuPeru_Primary.mdf',
    SIZE = 20MB,
    MAXSIZE = 200MB,
    FILEGROWTH = 10MB
),
FILEGROUP FG_Secundario (
    NAME = QhatuPeru_Secundario,
    FILENAME = 'C:\BD_QhatuPeru\Secundario\QhatuPeru_Secundario.ndf',
    SIZE = 10MB,
    MAXSIZE = 100MB,
    FILEGROWTH = 10MB
)
LOG ON (
    NAME = QhatuPeru_Log,
    FILENAME = 'C:\BD_QhatuPeru\Log\QhatuPeru_Log.ldf',
    SIZE = 10MB,
    MAXSIZE = 100MB,
    FILEGROWTH = 5MB
);
GO
```

	NombreArchivo	RutaFisica	TipoArchivo	Tamaño_MB
1	QhatuPeru_Primary	C:\BD_QhatuPeru\Primario\QhatuPeru_Primary.mdf	ROWS	20
2	QhatuPeru_Log	C:\BD_QhatuPeru\Log\QhatuPeru_Log.ldf	LOG	10
3	QhatuPeru_Secundario	C:\BD_QhatuPeru\Secundario\QhatuPeru_Secundario....	ROWS	10



Explicación de las buenas prácticas utilizadas

- Separación de archivos: los archivos primario, secundario y de log se guardaron en rutas distintas para distribuir la carga del disco.
- Nombres descriptivos: se emplearon nombres claros y coherentes para cada archivo físico y lógico, facilitando la administración.
- Control del crecimiento automático: mediante `FILEGROWTH` se evita la fragmentación del almacenamiento y se garantiza un desempeño estable.
- Uso de grupos de archivos (FILEGROUP): el archivo secundario se agregó a un grupo de archivos adicional para mejorar la organización del almacenamiento.
- Verificación de la estructura física: se consultó la vista `sys.database_files` para confirmar la correcta creación y ubicación de los archivos.
- Estructura de carpetas organizada: las rutas fueron previamente creadas en el sistema de archivos (`C:\BD_QhatuPeru\Primario\`, `Secundario\`, `Log\`), siguiendo estándares de organización y seguridad.

Ejercicio 2: Ajuste de configuración y validación de propiedades de la base de datos QhatuPeru

Enunciado del ejercicio

La empresa **QhatuPeru**, en su proceso de expansión al mercado nacional, solicita al equipo técnico realizar ajustes de configuración en su base de datos para optimizar el rendimiento y la compatibilidad lingüística.

Se requiere modificar la **colación** para que la base de datos soporte correctamente **tildes y caracteres especiales en español**, además de **configurar el crecimiento automático del archivo de datos primario** para evitar saturaciones inesperadas.

Ejercicio práctico:

- a) Consultar las propiedades actuales de la base de datos QhatuPeru, modificar la colación y configurar el crecimiento automático del archivo principal.
- b) Cambiar el crecimiento automático del archivo primario de datos a **20 MB**.

```

-- a) Consultar propiedades actuales de la base de datos
USE master;
GO
EXEC sp_helpdb 'QhatuPeru';
GO

-- a) Modificar la colación de la base de datos para soportar tildes y caracteres en español
ALTER DATABASE QhatuPeru
COLLATE Modern_Spanish_CI_AS;
GO

-- b) Configurar el crecimiento automático del archivo primario (a 20 MB)
ALTER DATABASE QhatuPeru
MODIFY FILE (
    NAME = QhatuPeru_Primary,
    FILEGROWTH = 20MB
);
GO

```

Results								Messages	
	name	db_size	owner	dbid	created	status		compatibility_level	
1	QhatuPeru	40.00 MB	LAB04-PC10\USER 17	5	Nov 6 2025	Status=ONLINE, Updateability=READ_WRITE, UserAcc...		160	

	name	fileid	filename	filegroup	size	maxsize	growth	usage
1	QhatuPeru_Primary	1	C:\BD_QhatuPeru\Primario\QhatuPeru_Primary.mdf	PRIMARY	20480 KB	204800 KB	10240 KB	data only
2	QhatuPeru_Log	2	C:\BD_QhatuPeru\Log\QhatuPeru_Log.ldf	NULL	10240 KB	102400 KB	5120 KB	log only
3	QhatuPeru_Secundario	3	C:\BD_QhatuPeru\Secundario\QhatuPeru_Secundario....	FG_Secundario	10240 KB	102400 KB	10240 KB	data only

	name	db_size	owner	dbid	created	status	compatibility_level
1	QhatuPeru	40.00 MB	LAB04-PC10\USER 17	5	Nov 6 2025	Status=ONLINE, Updateability=READ_WRITE, UserAcc...	160

	name	fileid	filename	filegroup	size	maxsize	growth	usage
1	QhatuPeru_Primary	1	C:\BD_QhatuPeru\Primario\QhatuPeru_Primary.mdf	PRIMARY	20480 KB	204800 KB	20480 KB	data only
2	QhatuPeru_Log	2	C:\BD_QhatuPeru\Log\QhatuPeru_Log.ldf	NULL	10240 KB	102400 KB	5120 KB	log only
3	QhatuPeru_Secundario	3	C:\BD_QhatuPeru\Secundario\QhatuPeru_Secundario....	FG_Secundario	10240 KB	102400 KB	10240 KB	data only

Justificación técnica de la solución aplicada

- Primero se consulta la colación y propiedades actuales para tener una referencia de la configuración previa.
- El comando `ALTER DATABASE ... COLLATE Modern_Spanish_CI_AS` permite cambiar la colación de la base de datos para que los datos en español (tildes, ñ, etc.) sean tratados correctamente. [Microsoft Learn+1](#)
- La sentencia `ALTER DATABASE ... MODIFY FILE ( ... FILEGROWTH = 20MB )` ajusta el crecimiento automático del archivo primario de datos para incrementar de forma más eficiente (20 MB por crecimiento) y evitar múltiples pequeñas expansiones que degradan el rendimiento.

- Finalmente se vuelve a consultar para verificar que los cambios fueron aplicados, lo cual es buena práctica en todo ajuste de configuración.

Explicación de las buenas prácticas utilizadas

- Consulta previa de configuración: verificar el estado antes de cambiar evita sorpresas y permite documentar el “antes” y “después”.
- Uso de colación adecuada para el idioma: la colación `Modern_Spanish_CI_AS` asegura que los caracteres especiales del español se traten correctamente.
- Crecimiento controlado del archivo: al definir un valor de crecimiento fijo (20 MB) se mejora la predictibilidad del almacenamiento y se reduce fragmentación.
- Verificación tras el cambio: ejecutar consultas después del ajuste es fundamental para confirmar que la nueva configuración está operativa.
- Separación de pasos claros (consulta, cambio, verificación): seguir este flujo estructurado ayuda a evitar errores en ambientes de producción.

Ejercicio 3: Definición de modelo de recuperación y respaldo para QhatuPeru

Enunciado del ejercicio

En el área de operaciones de QhatuPeru se identificaron diferentes necesidades de recuperación de datos según el proceso.

Se solicita configurar el modelo de recuperación adecuado y ejecutar un respaldo de la base de datos.

Ejercicio práctico:

- a) Cambia el modelo de recuperación de QhatuPeru a **SIMPLE** y luego a **BULK_LOGGED**, explicando la diferencia práctica entre ambos modelos.
- b) Realiza un **respaldo completo** después de cambiar al modelo **FULL**.

```
USE master;
GO

-- a) Ver el modelo de recuperación actual
SELECT name AS NombreBD,
       recovery_model_desc AS ModeloRecuperacion
FROM sys.databases
WHERE name = 'QhatuPeru';
GO

-- b) Cambiar al modelo SIMPLE
ALTER DATABASE QhatuPeru SET RECOVERY SIMPLE;
GO

-- Verificar el cambio
SELECT name AS NombreBD,
       recovery_model_desc AS ModeloRecuperacion
FROM sys.databases
WHERE name = 'QhatuPeru';
GO

-- c) Cambiar al modelo BULK_LOGGED
ALTER DATABASE QhatuPeru SET RECOVERY BULK_LOGGED;
GO

-- Verificar el cambio
SELECT name AS NombreBD,
       recovery_model_desc AS ModeloRecuperacion
FROM sys.databases
WHERE name = 'QhatuPeru';
GO

-- d) Cambiar al modelo FULL para realizar el respaldo completo
ALTER DATABASE QhatuPeru SET RECOVERY FULL;
GO

-- e) Realizar el respaldo completo de la base de datos
BACKUP DATABASE QhatuPeru
TO DISK = 'C:\BD_QhatuPeru\Backups\QhatuPeru_RespaldoCompleto.bak'
WITH INIT, NAME = 'Respaldo Completo QhatuPeru';
GO
```

	NombreBD	ModeloRecuperacion		NombreBD	ModeloRecuperacion
1	QhatuPeru	FULL	1	QhatuPeru	SIMPLE

	NombreBD	ModeloRecuperacion
1	QhatuPeru	BULK_LOGGED

 Backups
  QhatuPeru_RespaldoCompleto.bak

Antes de ejecutar el **BACKUP DATABASE**, asegúrate de que exista la carpeta:

C:\BD_QhatuPeru\Backups

Diferencia práctica entre los modelos **SIMPLE** y **BULK_LOGGED**

Modelo	Descripción	Ventajas	Desventajas
SIMPLE	No guarda registros detallados de transacciones; el log se trunca automáticamente.	Menor uso de espacio y mantenimiento más sencillo.	No permite restauraciones punto en el tiempo.
BULK_LOGGED	Registra solo operaciones críticas y de carga masiva; combina rendimiento y cierta seguridad.	Mejora el rendimiento en cargas grandes y permite restauración parcial.	Si ocurre un fallo durante una operación masiva, no se puede restaurar al punto exacto.

Ejercicio 4: Implementación de roles y usuarios para seguridad en QhatuPeru

Enunciado del ejercicio

Con el crecimiento de los equipos de ventas y atención al cliente, **QhatuPeru** solicita la creación de usuarios con roles diferenciados:

- Los **cajeros** solo pueden **consultar o registrar ventas**,
- Los **administradores** tienen **control total**, y
- El **gerente** requiere acceso únicamente a **reportes**.

Ejercicio práctico:

- a) Crear el usuario **VendedorQhatu** y asignarlo al rol **db_datawriter** para registrar ventas.
- b) Crear el usuario **ConsultaCliente** y asignarlo solo al rol **db_datareader** para consultas.

```
USE QhatuPeru;
GO

-- 1) Crear el inicio de sesión (Login) a nivel de servidor
CREATE LOGIN VendedorQhatu WITH PASSWORD = 'Vendedor#2025';
CREATE LOGIN ConsultaCliente WITH PASSWORD = 'Cliente#2025';
GO

-- 2) Crear los usuarios dentro de la base de datos QhatuPeru
CREATE USER VendedorQhatu FOR LOGIN VendedorQhatu;
CREATE USER ConsultaCliente FOR LOGIN ConsultaCliente;
GO

-- 3) Asignar roles a los usuarios según el perfil solicitado
-- Rol db_datawriter: puede insertar, actualizar y eliminar datos
ALTER ROLE db_datawriter ADD MEMBER VendedorQhatu;
GO

-- Rol db_datareader: solo puede consultar datos
ALTER ROLE db_datareader ADD MEMBER ConsultaCliente;
GO

-- 4) Verificar los roles asignados
EXEC sp_helpuser;
GO
```

Results		Messages					
	UserName	RoleName	LoginName	DefDBName	DefSchemaName	UserID	SID
1	ConsultaCliente	db_datareader	ConsultaCliente	master	dbo	6	0x9E7AE0D8635E4149820C134ADB43F0A2
2	dbo	db_owner	LAB04-PC10\USER 17	master	dbo	1	0x01050000000000051500000021B0D9FE4AF8C1C2C7657F...
3	guest	public	NULL	NULL	guest	2	0x00
4	INFORMATION_SCHEMA	public	NULL	NULL	NULL	3	NULL
5	sys	public	NULL	NULL	NULL	4	NULL
6	VendedorQhatu	db_datawriter	VendedorQhatu	master	dbo	5	0xC747881053A7984896702C8940BB0EA1

Justificación técnica de la solución aplicada

- Se utiliza **CREATE LOGIN** para crear los **accesos a nivel de servidor**, permitiendo que los usuarios puedan autenticarse en SQL Server.
- Se crea un **usuario interno de base de datos** con **CREATE USER** para asociar cada inicio de sesión (**LOGIN**) con la base de datos **QhatuPeru**.
- Se aplican roles integrados de SQL Server:
 - **db_datawriter**: permite realizar operaciones de inserción, actualización y eliminación (ideal para cajeros o vendedores).
 - **db_datareader**: permite consultar información sin modificarla (ideal para atención al cliente o reportes).
- Finalmente, se ejecuta **sp_helpuser** para verificar que las asignaciones se realizaron correctamente.

Explicación de las buenas prácticas utilizadas

- **Seguridad por roles**: se evita asignar permisos directamente a usuarios individuales, usando roles predefinidos que simplifican la administración.
- **Contraseñas seguras**: se definen contraseñas fuertes con mayúsculas, minúsculas y caracteres especiales.
- **Separación de privilegios**: cada usuario tiene solo los permisos necesarios para sus funciones.
- **Verificación posterior**: se valida la configuración con **sp_helpuser** para asegurar que todo quedó correctamente implementado.
- **Orden lógico de creación**: primero se crean los logins, luego los usuarios de base, y finalmente se asignan los roles.

Ejercicio 5: Configuración granular de permisos en el módulo de ventas de QhatuPeru

Enunciado del ejercicio

El **gerente de ventas** necesita revisar información del módulo de ventas **sin poder modificarla**.

El equipo técnico debe asignar permisos selectivos que garanticen el **principio de mínimo privilegio** dentro de la base de datos.

Ejercicio práctico:

- a) Otorga a **GerenteQhatu** acceso exclusivo (*solo SELECT*) a la tabla **Reportes**.
- b) Revoca a **CajeroQhatu** el permiso **UPDATE** sobre la tabla **Ventas**.

```
-- ☒ Otorgar permiso SOLO DE LECTURA a GerenteQhatu sobre Reportes
GRANT SELECT ON dbo.Reportes TO GerenteQhatu;
GO

-- ☒ Revocar permiso de UPDATE al usuario CajeroQhatu sobre Ventas
REVOKE UPDATE ON dbo.Ventas FROM CajeroQhatu;
GO

-- ☒ Consultar los permisos asignados para verificación
EXEC sp_helprotect @username = 'GerenteQhatu';
EXEC sp_helprotect @username = 'CajeroQhatu';
GO
```

Results		Messages					
	Owner	Object	Grantee	Grantor	ProtectType	Action	Column
1	dbo	Reportes	GerenteQhatu	dbo	Grant	Select	(All+New)
2	.	.	GerenteQhatu	dbo	Grant	CONNECT	.

	Owner	Object	Grantee	Grantor	ProtectType	Action	Column
1	.	.	CajeroQhatu	dbo	Grant	CONNECT	.

Justificación técnica

- Se crean los usuarios **GerenteQhatu** y **CajeroQhatu** dentro de la base QhatuPeru para aplicar permisos específicos sin afectar otros esquemas.
- Las tablas **Reportes** y **Ventas** se crean como ejemplos para simular el entorno real de ventas.

- Se usa **GRANT** para permitir acceso de lectura (solo **SELECT**) al gerente, y **REVOKE** para eliminar el permiso de actualización (**UPDATE**) al cajero, garantizando el **principio de mínimo privilegio**.
- **sp_helpprotect** permite verificar los permisos aplicados efectivamente a cada usuario.

Buenas prácticas aplicadas

- Uso de **IF NOT EXISTS** y **IF OBJECT_ID** para evitar errores si el objeto ya existe.
- Separación clara entre usuarios, roles y permisos, evitando sobreasignación.
- Se mantienen los permisos en el nivel **de tabla**, no global, cumpliendo políticas de seguridad granular.
- Scripts documentados y organizados con comentarios (**--**) para facilitar mantenimiento y auditoría.

Ejercicio 6: Identificación y solución de procesos lentos en QhatuPeru

1 Enunciado del ejercicio

Durante una campaña promocional, el sistema de QhatuPeru presenta **lentitud** en la ejecución de consultas.

El equipo técnico debe identificar los **procesos que más consumen CPU** y los **bloqueos activos** que puedan estar afectando el rendimiento.

Ejercicio práctico:

- a) Utiliza el **Activity Monitor** para identificar tres procesos que consuman más CPU durante una operación masiva.
- b) Consulta desde **T-SQL** los bloqueos actuales en la base de datos.

```
-- a) Identificar procesos con mayor consumo de CPU desde T-SQL
SELECT
    TOP 3
    r.session_id,
    r.status,
    r.command,
    s.login_name,
    s.host_name,
    r.cpu_time AS TiempoCPU_ms,
    r.total_elapsed_time / 1000 AS TiempoTranscurrido_seg,
    DB_NAME(r.database_id) AS BaseDatos,
    SUBSTRING(t.text, (r.statement_start_offset/2)+1,
        ((CASE r.statement_end_offset
            WHEN -1 THEN DATALENGTH(t.text)
            ELSE r.statement_end_offset END
            - r.statement_start_offset)/2)+1) AS ConsultaEjecutada
FROM sys.dm_exec_requests r
INNER JOIN sys.dm_exec_sessions s ON r.session_id = s.session_id
CROSS APPLY sys.dm_exec_sql_text(r.sql_handle) AS t
WHERE r.session_id <> @@SPID
ORDER BY r.cpu_time DESC;
GO

-- b) Consultar bloqueos actuales en la base de datos
SELECT
    bl.request_session_id AS SesionBloqueada,
    wt.blocking_session_id AS SesionBloqueante,
    OBJECT_NAME(p.object_id) AS TablaAfectada,
    p.index_id,
    wt.wait_type,
    wt.wait_duration_ms,
    wt.resource_description
FROM sys.dm_tran_locks AS bl
INNER JOIN sys.dm_os_waiting_tasks AS wt
    ON bl.lock_owner_address = wt.resource_address
INNER JOIN sys.partitions AS p
    ON p.hobt_id = bl.resource_associated_entity_id
WHERE bl.resource_database_id = DB_ID('QhatuPeru');
GO
```

session_id	status	command	login_name	host_name	TiempoCPU_ms	TiempoTranscurrido_seg	BaseDatos	ConsultaEjecutada
------------	--------	---------	------------	-----------	--------------	------------------------	-----------	-------------------

SesionBloqueada	SesionBloqueante	TablaAfectada	index_id	wait_type	wait_duration_ms	resource_description
-----------------	------------------	---------------	----------	-----------	------------------	----------------------

Justificación técnica

- La vista dinámica **sys.dm_exec_requests** permite visualizar las consultas activas, su tiempo de CPU y duración total, identificando los procesos que más carga generan.
- Se usa **sys.dm_exec_sql_text** para obtener el texto exacto de la consulta ejecutada.
- Para detectar bloqueos, se relacionan las DMV (Dynamic Management Views) **sys.dm_tran_locks**, **sys.dm_os_waiting_tasks** y **sys.partitions**, lo que facilita identificar **qué sesión bloquea a otra y qué tabla está involucrada**.
- Este análisis ayuda al DBA a determinar si la lentitud proviene de **consultas mal optimizadas**, **índices ausentes** o **bloqueos entre transacciones**.

Buenas prácticas aplicadas

- Se usa **TOP 3** para obtener solo los procesos más relevantes y evitar sobrecargar la consulta.
- Se excluye la sesión actual (**WHERE r.session_id <> @@SPID**) para no distorsionar resultados.
- Las DMV utilizadas son herramientas estándar de diagnóstico de SQL Server, recomendadas por Microsoft.
- El uso del **principio de observación no intrusiva** permite monitorear el rendimiento sin afectar la base.
- Los resultados deben complementarse con el uso de **Activity Monitor** desde SQL Server Management Studio para correlacionar visualmente el consumo de CPU y bloqueos.

Ejercicio 7: Automatización de respaldos y limpieza del sistema QhatuPeru

Enunciado del ejercicio

La empresa **QhatuPeru** requiere automatizar tareas básicas de mantenimiento del sistema, como la **generación diaria de respaldos** y la **limpieza semanal de registros antiguos**.

Estas tareas deben configurarse mediante **Jobs en SQL Server Agent**, para ejecutarse sin intervención manual.

Ejercicio práctico:

- a) Crea un Job en SQL Server Agent que realice un **respaldo diario automático** de QhatuPeru.
- b) Diseña y programa un Job que **elimine registros** de la tabla **Sesiones** con más de 15 días de antigüedad, ejecutándose **cada semana**.

```
USE msdb;
GO

-- ✅ Crear el Job de respaldo diario
EXEC sp_add_job
    @job_name = N'Respaldo_Diario_QhatuPeru',
    @description = N'Realiza un respaldo completo diario de la base de datos QhatuPeru';
GO

-- ✅ Crear el paso del Job (respaldo completo)
EXEC sp_add_jobstep
    @job_name = N'Respaldo_Diario_QhatuPeru',
    @step_name = N'Realizar respaldo completo',
    @subsystem = N'TSQL',
    @command = N'
        BACKUP DATABASE QhatuPeru
        TO DISK = ''C:\BD_QhatuPeru\Backups\QhatuPeru_FullBackup.bak''
        WITH FORMAT, INIT, NAME = ''Respaldo completo diario de QhatuPeru'', SKIP, STATS = 10;';
    @database_name = N'master';
GO

-- ✅ Programar ejecución diaria (cada día a las 23:00)
EXEC sp_add_schedule
    @schedule_name = N'Schedule_Respaldo_Diario',
    @freq_type = 4,           -- Diario
    @freq_interval = 1,       -- Cada 1 día
    @active_start_time = 230000; -- 23:00 horas
GO

-- ✅ Asociar el schedule al Job
EXEC sp_attach_schedule
    @job_name = N'Respaldo_Diario_QhatuPeru',
    @schedule_name = N'Schedule_Respaldo_Diario';
GO

-- ✅ Habilitar el Job
EXEC sp_add_jobserver
    @job_name = N'Respaldo_Diario_QhatuPeru';
GO
```

SQLServerAgent is not currently running so it cannot be notified of this action.

Completion time: 2025-11-06T10:44:56.6287562-05:00

```

USE msdb;
GO

-- ☒ Crear Job de limpieza semanal
EXEC sp_add_job
    @job_name = N'Limpieza_Semanal_Sesiones',
    @description = N'Elimina registros de la tabla Sesiones con más de 15 días de antigüedad';
GO

-- ☒ Crear el paso del Job
EXEC sp_add_jobstep
    @job_name = N'Limpieza_Semanal_Sesiones',
    @step_name = N'Eliminar registros antiguos',
    @subsystem = N'TSQL',
    @command = N'
        USE QhatuPeru;
        IF OBJECT_ID(''dbo.Sesiones'') IS NOT NULL
        BEGIN
            DELETE FROM dbo.Sesiones
            WHERE DATEDIFF(DAY, FechaSesion, GETDATE()) > 15;
        END;',
    @database_name = N'QhatuPeru';
GO

-- ☒ Programar ejecución semanal (domingo a las 02:00 a.m.)
EXEC sp_add_schedule
    @schedule_name = N'Schedule_Limpieza_Semanal',
    @freq_type = 8,           -- 8 = Semanal
    @freq_interval = 1,      -- 1 = Domingo
    @freq_recurrence_factor = 1, -- Cada 1 semana
    @active_start_time = 20000; -- 02:00 a.m.
GO

-- ☒ Asociar el schedule al Job
EXEC sp_attach_schedule
    @job_name = N'Limpieza_Semanal_Sesiones',
    @schedule_name = N'Schedule_Limpieza_Semanal';
GO

-- ☒ Habilitar el Job (se requiere que SQL Server Agent esté activo)
EXEC sp_add_jobserver
    @job_name = N'Limpieza_Semanal_Sesiones';
GO

```

Justificación técnica

- Los **Jobs** permiten automatizar tareas repetitivas mediante el **SQL Server Agent**, mejorando la eficiencia del mantenimiento.
- El primer Job garantiza la **disponibilidad de respaldos actualizados**, reduciendo el riesgo de pérdida de información.
- El segundo Job optimiza el rendimiento de la base al eliminar datos antiguos, cumpliendo políticas de retención de información.
- Se utilizan los procedimientos almacenados del sistema (`sp_add_job`, `sp_add_jobstep`, `sp_add_schedule`, etc.), que aseguran compatibilidad con SQL Server.

Buenas prácticas aplicadas

- Uso de **rutas absolutas** y directorios predefinidos para el almacenamiento de backups (C:\BD_QhatuPeru\Backups).
- Programación de tareas fuera del horario laboral para minimizar impacto en el sistema (respaldo a las **23:00**, limpieza a las **02:00**).
- Eliminación controlada mediante **DATEDIFF** garantiza que solo se borren registros realmente antiguos.
- Estructura modular: cada Job tiene nombre, descripción, paso, y schedule claros.
- Supervisión periódica recomendada desde SQL Server Agent > Jobs > History para confirmar ejecución exitosa.

Ejercicio 8: Registro de estados en pedidos en QhatuPeru

Enunciado del ejercicio

Con la finalidad de optimizar la trazabilidad de los envíos, se solicita agregar una columna **EstadoEnvio** en la tabla **Pedidos**, permitiendo identificar el estado actual de cada pedido.

Ejercicio práctico:

- Agrega la columna **Prioridad** tipo **INT** a la tabla **Pedidos**.
- Elimina la columna **EstadoEnvio** de la tabla **Pedidos**.

```
USE QhatuPeru;
GO

-- a) Agregar columna 'Prioridad' tipo INT
ALTER TABLE Pedidos
ADD Prioridad INT;
GO

-- Verificar que se haya agregado correctamente
EXEC sp_columns Pedidos;
GO

-- b) Eliminar columna 'EstadoEnvio'
ALTER TABLE Pedidos
DROP COLUMN EstadoEnvio;
GO

-- Confirmar estructura final de la tabla
EXEC sp_columns Pedidos;
GO
```

	TABLE_QUALIFIER	TABLE_OWNER	TABLE_NAME	COLUMN_NAME	DATA_TYPE	TYPE_NAME	PRECISION	LENGTH	SCALE	RADIX	NULLABLE	REMARKS	COLUMN_DEF	SQL_DATA_TYPE	SQL_DATETIME_SUB	CHAR_OCTET_LENGTH	ORDINAL_POSITION	IS_NULLABLE
1	QhatuPeru	dbo	Pedidos	IdPedido	int	identity	10	4	0	10	0	NULL	NULL	4	NULL	NULL	1	NO
2	QhatuPeru	dbo	Pedidos	FechaPedido	-9	date	10	20	NULL	NULL	0	NULL	NULL	-9	NULL	NULL	2	NO
3	QhatuPeru	dbo	Pedidos	Cliente	-9	nvarchar	100	200	NULL	NULL	0	NULL	NULL	-9	NULL	200	3	NO
4	QhatuPeru	dbo	Pedidos	Monto	3	decimal	10	12	2	10	0	NULL	NULL	3	NULL	NULL	4	NO
5	QhatuPeru	dbo	Pedidos	EstadoEnvio	-9	nvarchar	50	100	NULL	NULL	1	NULL	NULL	-9	NULL	100	5	YES
6	QhatuPeru	dbo	Pedidos	Prioridad	4	int	10	4	0	10	1	NULL	NULL	4	NULL	NULL	6	YES

Justificación técnica

El uso de los comandos ALTER TABLE ... ADD y ALTER TABLE ... DROP COLUMN permite modificar la estructura de una tabla existente sin necesidad de recrearla.

Estos cambios son seguros y eficientes cuando se aplican correctamente, ya que preservan los datos existentes y solo ajustan la definición del esquema.

Agregar la columna **Prioridad** permite asignar un nivel de importancia a cada pedido, facilitando la gestión operativa.

La eliminación de **EstadoEnvio** se realiza para mantener la base de datos limpia y alineada con los nuevos requerimientos funcionales.

Buenas prácticas aplicadas

- Verificación previa con **sp_columns** antes y después de los cambios para asegurar que las modificaciones se ejecutaron correctamente.
- Ejecución dentro del contexto de la base de datos con **USE QhatuPeru** para evitar errores de referencia.
- Uso de tipos de datos adecuados (**INT** para prioridad, permitiendo escalabilidad).
- Documentación clara de cada paso (comentarios antes de cada instrucción).
- Control de impacto: se recomienda realizar estos cambios en horarios de baja carga o en un entorno de pruebas previo a producción.

Ejercicio 9: Implementación de registros automáticos de modificaciones en QhatuPeru

Enunciado del ejercicio

Es necesario auditar todas las modificaciones realizadas en la tabla **Cientes** para cumplir con políticas internas de seguridad y protección de datos.

Ejercicio práctico:

- a) Crear una tabla **AuditoriaCientes** para registrar los cambios en **Cientes**.
- b) Desarrollar un **trigger** que registre en **AuditoriaCientes** cada eliminación de registros en **Cientes**.

```
-- a) Crear tabla base si no existe
IF OBJECT_ID('Cientes') IS NULL
BEGIN
    CREATE TABLE Cientes (
        IdCliente INT IDENTITY(1,1) PRIMARY KEY,
        Nombre NVARCHAR(100),
        Correo NVARCHAR(100),
        Telefono NVARCHAR(20)
    );
END
GO

-- Crear tabla de auditoria
CREATE TABLE AuditoriaCientes (
    IdAuditoria INT IDENTITY(1,1) PRIMARY KEY,
    IdCliente INT,
    Nombre NVARCHAR(100),
    Correo NVARCHAR(100),
    Telefono NVARCHAR(20),
    FechaEliminacion DATETIME DEFAULT GETDATE(),
    Usuario NVARCHAR(100)
);
GO

-- b) Crear trigger para registrar eliminaciones
CREATE TRIGGER trg_Cientes_Delete
ON Cientes
AFTER DELETE
AS
BEGIN
    INSERT INTO AuditoriaCientes (IdCliente, Nombre, Correo, Telefono, Usuario)
    SELECT
        d.IdCliente,
        d.Nombre,
        d.Correro,
        d.Telefono,
        SUSER_SNAME() -- usuario que ejecutó la acción
    FROM deleted d;
END;
GO

-- Probar el trigger (opcional)
INSERT INTO Cientes (Nombre, Correo, Telefono)
VALUES ('Juan Pérez', 'juanperez@mail.com', '999999999');

DELETE FROM Cientes WHERE Nombre = 'Juan Pérez';

SELECT * FROM AuditoriaCientes;
GO
```

Resultados		Mensajes					
	IdAuditoria	IdCliente	Nombre	Correo	Telefono	FechaEliminacion	Usuario
1	1	1	Juan Pérez	juanperez@mail.com	999999999	2025-11-06 13:42:38.113	DESKTOP-KUFV7TG\Daniel

Justificación técnica de la solución aplicada

- Se crea la tabla **AuditoriaClientes** para **almacenar evidencia de eliminaciones**, incluyendo fecha y usuario responsable.
- El **trigger AFTER DELETE** permite interceptar las operaciones de eliminación y registrar automáticamente la información del registro eliminado.
- Se usa la tabla lógica **deleted**, que almacena temporalmente los registros eliminados dentro del contexto del trigger.
- **SUSER_SNAME()** obtiene el nombre del usuario de SQL Server que ejecutó la acción, útil para trazabilidad y cumplimiento normativo.

Buenas prácticas aplicadas

- **Creación condicional** de la tabla **Clientes** garantiza que el script sea reutilizable sin errores.
- Se incluye una **columna de fecha con valor por defecto (GETDATE())** para registrar automáticamente el momento de la eliminación.
- Se emplea **nomenclatura clara y estandarizada (trg_Clientes_Delete)** que describe el propósito del trigger.
- El trigger registra únicamente eliminaciones, aplicando el **principio de mínima intervención** sobre el sistema.
- Se verifica el correcto funcionamiento del trigger mediante una prueba controlada.

Ejercicio 10: Simulación de restauración tras un incidente en QhatuPeru

Enunciado del ejercicio

Por un error humano se eliminó información de la tabla **Cientes**.

El equipo debe restaurar la base usando el **respaldo más reciente** y validar la recuperación exitosa de los datos.

Ejercicio práctico:

- a) Simular la eliminación de registros de la tabla **Cientes** y la posterior restauración.
- b) Verificar la existencia y correcta restauración de los datos desde el backup.

```
USE QhatuPeru;
GO

-- ☒ Crear respaldo completo (si no existe)
BACKUP DATABASE QhatuPeru
TO DISK = 'C:\BD_QhatuPeru\Backups\QhatuPeru_Full.bak'
WITH INIT,
    NAME = 'Backup completo QhatuPeru';
GO

-- ☒ Simular eliminación accidental de datos
DELETE FROM Cientes;
GO

-- Comprobar que está vacía
SELECT * FROM Cientes;
GO

-- ☒ Restaurar base desde el respaldo más reciente
USE master;
GO
RESTORE DATABASE QhatuPeru
FROM DISK = 'C:\BD_QhatuPeru\Backups\QhatuPeru_Full.bak'
WITH REPLACE, RECOVERY;
GO

-- ☒ Verificar recuperación exitosa
USE QhatuPeru;
GO
SELECT * FROM Cientes;
GO
```

Justificación técnica de la solución aplicada

- El **respaldo completo** garantiza una copia íntegra de la base de datos en un punto seguro antes de cualquier incidente.
- La instrucción **DELETE** permite **simular un error humano real**, útil para validar los procedimientos de recuperación.
- **RESTORE DATABASE ... WITH REPLACE** sobrescribe la base actual con la versión del respaldo, asegurando una **recuperación total** de los datos eliminados.
- **RECOVERY** deja la base **operativa inmediatamente** tras la restauración, lista para consultas y operaciones normales.
- Se realiza una consulta posterior (**SELECT * FROM Clientes**) para **validar la integridad y recuperación de la información**.

Buenas prácticas aplicadas

- **Rutas claras y organizadas** para los respaldos (**C:\BD_QhatuPeru\Backups**) facilitan la administración y evitan pérdidas.
- Mantener **backups frecuentes** (diarios o automáticos mediante SQL Agent) asegura un punto de restauración reciente.
- Uso del modificador **WITH INIT** para **evitar duplicados o archivos corruptos** en la misma ruta.
- Se realiza la restauración **desde el entorno master**, práctica recomendada para no bloquear la base en uso.
- Validación final garantiza cumplimiento con **políticas de seguridad y continuidad operativa**.